İTÜ ROVER TAKIMI
ELEKTRONİK EKİBİ

2026 Gömülü Sistemler Ödevi 6. Kodlama

UART mesajlarıyla iki GPIO çıkışının kontrolü

STM32F446RET6 / NUCLEO-F446RE
STM32CubeIDE 2.2.0 / STM32F4 HAL

Teslim bileşeni	Durum
HAL uygulaması	Satır açıklamalı C kaynakları ve içe aktarılabilir CubeIDE projesi
Derleme	Debug: 0 hata, 0 uyarı
Yazılım doğrulaması	86 ARM emülasyon senaryosu başarılı
Fiziksel doğrulama	Kart üzerinde yükleme ve gerilim ölçümü yapılmadı
GitHub	Ayrı proje deposu; bağlantı raporun 6. sayfasında

Hazırlanma tarihi: 7 Ekim 2026

Bu rapor yalnızca ödevin 6. Kodlama bölümünü kapsar. Diğer teorik bölümler bu belgenin kapsamında değildir.

1. Amaç, kart seçimi ve bağlantılar

Amaç, bilgisayardan UART üzerinden gelen iki karakterli bir mesajı doğrulayıp iki ayrı dijital çıkışa LOW, HIGH veya TOGGLE işlemi uygulamaktır. İlk karakter PA5, ikinci karakter PA6 için kullanılır. Başlangıçta iki çıkış LOW yapılır.

Neden STM32F446RE?

NUCLEO-F446RE kartındaki programlayıcı/hata ayıklayıcı ve Virtual COM bağlantısı, bu ödevde tek USB bağlantısıyla geliştirme ve seri iletişim kurulmasını kolaylaştırır. PA5 üzerindeki kullanıcı LED'i birinci çıkışı gözlemlemeyi sağlar. Seçim, işlem gücünden çok geliştirme kolaylığına dayanır. Kart özellikleri ve bağlantılar ST ürün sayfası ile UM1724 kılavuzundan doğrulanmıştır [1, 2].

Görev	STM32 pini	Kart bağlantısı
Birinci çıkış	PA5	Arduino D13, CN5 pin 6; LD2 LED
İkinci çıkış	PA6	Arduino D12, CN5 pin 5
USART2 gönderme	PA2 / AF7	ST-Link Virtual COM hattı
USART2 alma	PA3 / AF7	ST-Link Virtual COM hattı
Referans	GND	Harici ölçüm/LED için ortak toprak

Fiziksel kurulum

Bilgisayar, veri taşıyan USB kablosuyla kartın ST-Link USB girişine bağlanır. Windows Aygıt Yöneticisi'nde oluşan Virtual COM portu terminalde seçilir. PA6 için multimetre veya lojik analizör kullanılabilir; görsel kontrol için PA6 - 330 ohm direnç - LED anot bağlantısı yapılır, LED katodu GND'ye bağlanır.

Bilgisayar	Kart içi bağlantı	Çıkışlar
Terminal / USB	ST-Link VCP → USART2 → mesaj ayrıştırıcı	PA5 → LD2 PA6 → harici LED / ölçüm

GPIO mantık seviyeleri nominal olarak LOW ≈ 0 V ve HIGH ≈ 3,3 V'tur; bunlar ölçüm sonucu değildir. Motor gibi güç yükleri GPIO'ya doğrudan bağlanmaz. Kart değiştirilmişse veya revizyon farklıysa Virtual COM yönlendirmesi ve lehim köprüleri UM1724'e göre kontrol edilmelidir [2].

2. IDE ve HAL yapılandırmaları

Proje, ST'nin resmi STM32CubeF4 F446RE CubeIDE şablonundan türetilmiştir [4]. Kaynaklar ve gerekli sürücüler proje içine alınmıştır. Aşağıdaki ayarlar gerçek .cproject dosyasında ve HAL başlangıç fonksiyonlarında uygulanmıştır; tablo bir ekran görüntüsü değildir.

Alan	Uygulanan değer
Hedef / dil	STM32F446RETx / C (GNU11)
Derleme	Debug, -O0, Cortex-M4, Thumb, FPv4-SP-D16, hard-float
Önişlemci	USE_HAL_DRIVER, STM32F446xx, DEBUG
Linker	STM32F446RETX_FLASH.ld
Saat kaynağı	HSI 16 MHz, PLL kapalı
Veri yolu saatleri	SYSCLK = HCLK = PCLK1 = PCLK2 = 16 MHz
Flash / zaman tabanı	Latency 0; SysTick 1 ms
PA5 ve PA6	GPIO_Output, Push-Pull, No Pull, Low Speed, ilk değer LOW
PA2 ve PA3	AF7 USART2, Push-Pull, Pull-Up, Very High Speed
USART2	Asynchronous UART, 115200 baud, 8 bit, parity None, 1 stop bit
Akış / alım yöntemi	Flow control None; polling, 1 bayt, 10 ms bekleme
Kesme / DMA / RTOS	SysTick aktif; USART2 IRQ, DMA ve RTOS kullanılmaz

Projeyi açma

CubeIDE'de File > Import > General > Existing Projects into Workspace seçilir; Rover_UART_GPIO klasörü gösterilir. Project > Build Project ile Debug derlenir. Sürücüler proje içindedir; başka bir bilgisayarda kullanıcının STM32Cube/Repository dizinine ihtiyaç duyulmaz.

CubeIDE 2.x'te CubeMX ayrı bir uygulamadır [5]. Bu teslimde .ioc veya grafik yapılandırma ekran görüntüsü üretilmemiştir. Grafik arayüzle yeniden kurmak için aynı hedef, pin ve saat değerleri ayrı STM32CubeMX uygulamasında seçilip Toolchain/IDE = STM32CubeIDE olarak üretilebilir. Üretilen iskeletle buradaki özel kaynaklar birleştirilirken yinelenen main/MSP/kesme tanımları kaldırılmalıdır.

3. UART mesaj biçimi ve çalışma akışı

Terminal 115200 baud, 8N1, flow control None ve ASCII gönderimiyle kullanılır. Her mesaj iki karakterden sonra Enter ile tamamlanır: örneğin **21 + CR/LF**. İlk karakter PA5'i, ikinci karakter PA6'yı yönetir. Satır sonu zorunludur; yalnızca iki karakter yazmak çıkışları hemen değiştirmez.

Mesaj	PA5 / birinci çıkış	PA6 / ikinci çıkış
00	LOW	LOW
01	LOW	HIGH
02	LOW	TOGGLE
10	HIGH	LOW
11	HIGH	HIGH
12	HIGH	TOGGLE
20	TOGGLE	LOW
21	TOGGLE	HIGH
22	TOGGLE	TOGGLE

TOGGLE, çıkış mandalının önceki durumunu tersine çevirir. Örneğin PA5 LOW iken 21 gönderilirse PA5 HIGH olur; aynı mesaj tekrar gönderilirse PA5 LOW olur. İkinci çıkış her iki durumda HIGH kalır.

Mesaj doğrulama

Alınan karakterler iki baytlık tamponda biriktirilir. CR veya LF geldiğinde uzunluk tam iki ve karakterler 0/1/2 ise iki komut uygulanır. Eksik mesaj, üçüncü karakter, boşluk veya başka karakter bütün satırı geçersiz yapar. Çıkışlar değiştirilmeden yeni satır beklenir. CRLF içindeki ikinci satır sonu boş satır olduğundan TOGGLE tekrar uygulanmaz.

Bekleyen mesajda bir saniyeden uzun karakter aralığı oluşursa tampon sıfırlanır. UART alım hatası biriktirilmiş çerçeveyi iptal eder. İki pin sıralı HAL çağrılarıyla güncellenir; aynı saat çevriminde eşzamanlı değişim garantisi yoktur. Bu basit uygulamada ACK/echo gönderilmez; sonuç LED veya ölçümle gözlenir.

4. Kodun yapısı ve doğrulama

Dosya	Görevi
main.c	HAL başlatma, saat/GPIO/UART ayarı, alım döngüsü, pin komutları
command.c / command.h	Bağımsız mesaj ayrıştırıcı; tam ve geçerli mesajı çıkarma
stm32f4xx_hal_msp.c	USART2 saatleri, PA2/PA3 alternatif fonksiyon ayarları
stm32f4xx_it.c	SysTick ile HAL zaman sayacını artırma
ST destek dosyaları	Startup, system, syscalls/sysmem ve HAL/CMSIS sürücüleri

Yeni yazılan uygulama kaynaklarının her satırı Türkçe açıklamayla yorumlanmıştır. Kod yorumlarında kaynak dosyası taşınabilirliği için ASCII karakterler kullanılmıştır. ST tarafından sağlanan destek ve sürücü dosyaları özgün açıklama ve telif bildirimleriyle korunmuştur. Yeni uygulama kodu Ek A'da verilmiştir.

Gerçekleşen kontroller

Kurulu STM32CubeIDE 2.2.0 headless build uygulaması ile proje içe aktarılıp Debug olarak derlenmiştir. build_success.txt günlüğü **0 errors, 0 warnings** sonucu içerir. ELF oluşturulmuş; ayrıca HEX ve BIN çıktıları hazırlanmıştır. Derleme boyutu: text 9448 bayt, data 12 bayt, bss 1644 bayt. bss değeri ayrılmış heap/stack alanlarını da içerir.

Gerçek command.c ve main.c içindeki ApplyPin fonksiyonu Cortex-M4 makine koduna derlenip Unicorn ARM emülatöründe çalıştırılmıştır. HAL GPIO fonksiyonları, pin durumunu izleyen test doubles ile temsil edilmiştir. **86 senaryo başarılıdır.** Bu yöntem ayrıştırıcıyı ve GPIO komut seçimini sınar; fiziksel GPIO, UART alım döngüsü, zaman aşımı ve osilatör davranışını donanımda doğrulamaz.

Test grubu	Sayı	Sonuç
9 komut × 4 başlangıç durumu	36	PASS
11 geçersiz/eksik girdi × 4 durum	44	PASS
Toparlanma, ardışık toggle, CR/LF ve karışık sıra	6	PASS
Fiziksel kart / UART / GPIO ölçümü	Yapılmadı	Bekleyen donanım doğrulaması

Emülasyon testinde assert ile beklenen pin durumları karşılaştırılır. test_results.json tam girdileri ve sonuçları içerir. Kaynak kodun aynısı test edildiğinden farklı bir Python ayrıştırıcı modelinin doğru çalışması test sonucu olarak kullanılmamıştır.

5. Yükleme, donanım testi ve kaynaklar

ST-Link ile yükleme ve terminal

Kart bağlıyken Run > Debug Configurations > STM32 C/C++ Application altında ST-Link ve SWD seçilir. ELF yolu Debug/Rover_UART_GPIO.elf olmalıdır. Debug ile program Flash'a yüklenir; Resume ile main döngüsü çalıştırılır. Terminalde kartın COM portu açılır. PuTTY için local echo / local line editing kullanılabilir. Hercules Send alanında ASCII mesajın sonuna CR/LF eklenir [3].

Kart üzerinde uygulanacak test

Önce reset sonrası iki çıkışın LOW olduğu doğrulanır. Her senaryo öncesi 00 + Enter gönderilir; dokuz komut ayrı ayrı gönderilerek PA5/PA6 seviyeleri ölçülür. Başlangıç 00 olduğunda 2 komutu ilgili pini HIGH yapmalıdır. Ardından 22 iki kez gönderilerek iki pinin önce HIGH, sonra LOW olduğu görülmelidir. 111, x21, tek karakter ve boş satır çıkışları değiştirmemelidir.

Karakter aralığı testi için ilk karakter gönderilip bir saniyeden uzun beklenir; ikinci karakter ve Enter gönderildiğinde eksik çerçeve uygulanmamalıdır. Kart üzerindeki LD2 yalnızca PA5'i gösterir; PA6 mutlaka ayrıca ölçülmelidir. Gerçek ölçüm sonuçları alındığında tarih, komut, başlangıç ve gözlenen durumlar rapora eklenebilir. Bu teslimde bu ölçümler yapılmamıştır.

GitHub teslimi

https://github.com/livintragedy/itu-rover-uart-gpio-2026

Depo; içe aktarılabilir CubeIDE projesi, satır açıklamalı kaynaklar, README, derleme günlüğü, emülasyon testleri, firmware çıktıları ve bu PDF raporunu içerir. Repo bağlantısı raporun zorunlu GitHub alanını karşılar.

Kaynakça - erişim: 7 Ekim 2026

1. ST NUCLEO-F446RE ürün bilgisi

2. ST UM1724, Nucleo-64 MB1136 kullanıcı kılavuzu

3. ST UART başlangıç rehberi

4. ST resmi STM32CubeF4 kaynak deposu

5. ST CubeIDE 2.0 sürüm duyurusu

Ek A. Satır açıklamalı uygulama kodu

Aşağıdaki liste teslim edilen kaynak dosyalarından doğrudan üretilmiştir. Uzun satırlar PDF'de görsel olarak bölünmüştür; soldaki numara kaynak satırını belirtir. Kaynak dosyalarında satır numaraları yoktur.

Core/Inc/command.h

```
 1  #ifndef COMMAND_H /* Basligin birden fazla eklenmesini onler. */
 2  #define COMMAND_H /* Baslik korumasini etkinlestirir. */
 3  #include <stdint.h> /* Sabit genislikli tamsayi turlerini saglar. */
 4  typedef struct { /* UART mesajinin biriktirme durumunu tanimlar. */
 5      uint8_t data[2]; /* Iki GPIO icin iki komut karakterini tutar. */
 6      uint8_t length; /* Simdiye kadar alinan karakter sayisini tutar. */
 7      uint8_t invalid; /* Hatali veya fazla karakterli mesaji isaretler. */
 8  } CommandParser; /* Yapiya kullanisli bir tur adi verir. */
 9  void Command_Reset(CommandParser *p); /* Yeni mesaj icin durumu sifirlar. */
10  uint8_t Command_Feed(CommandParser *p, uint8_t byte, uint8_t out[2]); /* Gecerli tamamlanmis mesajda 1
    doner. */
11  #endif /* Baslik korumasini sonlandirir. */
```

Core/Src/command.c

```c
#include "command.h" /* Mesaj ayristiricisinin tur ve bildirimlerini alir. */
void Command_Reset(CommandParser *p) /* Mesaj durumunu temizleyen fonksiyonu tanimlar. */
{ /* Fonksiyon govdesini baslatir. */
    p->length = 0U; /* Biriktirilmis karakter sayisini sifirlar. */
    p->invalid = 0U; /* Hatali mesaj bayragini temizler. */
} /* Sifirlama fonksiyonunu bitirir. */
uint8_t Command_Feed(CommandParser *p, uint8_t byte, uint8_t out[2]) /* Tek UART baytini isler. */
{ /* Ayristirma fonksiyonunu baslatir. */
    if ((byte == '\r') || (byte == '\n')) /* CR veya LF geldiginde mesaj sonunu algilar. */
    { /* Mesaj sonu dalini baslatir. */
        uint8_t ready = (p->length == 2U) && (p->invalid == 0U); /* Yalnizca iki gecerli karakteri kabul eder. */
        if (ready != 0U) /* Gecerli mesaj varsa cikti tamponunu doldurur. */
        { /* Kopyalama blogunu baslatir. */
            out[0] = p->data[0]; /* Birinci pinin komutunu aktarir. */
            out[1] = p->data[1]; /* Ikinci pinin komutunu aktarir. */
        } /* Kopyalama blogunu bitirir. */
        Command_Reset(p); /* Sonraki mesaj icin durumu temizler. */
        return ready; /* Mesajin uygulanabilir olup olmadigini bildirir. */
    } /* Mesaj sonu dalini bitirir. */
    if ((byte < '0') || (byte > '2') || (p->length >= 2U)) /* Gecersiz karakteri ve fazla uzun mesaji saptar. */
    { /* Hata dalini baslatir. */
        p->invalid = 1U; /* Satir sonuna kadar tum mesaji reddeder. */
    } /* Hata dalini bitirir. */
    else /* Tamponda yer ve gecerli karakter varsa bu dali kullanir. */
    { /* Biriktirme dalini baslatir. */
        p->data[p->length++] = byte; /* Karakteri kaydeder ve sayaci artirir. */
    } /* Biriktirme dalini bitirir. */
    return 0U; /* Satir sonu gelmedigi icin henuz komut uygulamaz. */
} /* Ayristirma fonksiyonunu bitirir. */
```

Core/Src/main.c

```
#include "stm32f4xx_hal.h" /* STM32F4 HAL fonksiyon ve turlerini ekler. */
#include "command.h" /* Iki karakterli mesaj ayristiricisini ekler. */
UART_HandleTypeDef huart2; /* USART2 donaniminin HAL durumunu tutar. */
static void Error_Handler(void); /* Baslatma hatasi fonksiyonunu bildirir. */
static void SystemClock_Config(void); /* Sistem saatini ayarlayan fonksiyonu bildirir. */
static void MX_GPIO_Init(void); /* GPIO baslatma fonksiyonunu bildirir. */
static void MX_USART2_UART_Init(void); /* UART baslatma fonksiyonunu bildirir. */
static void ApplyPin(uint16_t pin, uint8_t command) /* Tek cikis pinine komut uygular. */
{ /* Pin kontrol fonksiyonunu baslatir. */
    if (command == '2') /* TOGGLE komutunu saptar. */
    { /* TOGGLE dalini baslatir. */
        HAL_GPIO_TogglePin(GPIOA, pin); /* Cikis mandalinin mevcut degerini tersler. */
    } /* TOGGLE dalini bitirir. */
    else /* LOW veya HIGH komutunu ele alir. */
    { /* Yazma dalini baslatir. */
        HAL_GPIO_WritePin(GPIOA, pin, command == '1' ? GPIO_PIN_SET : GPIO_PIN_RESET); /* 1 icin HIGH, 0
 icin LOW yazar. */
    } /* Yazma dalini bitirir. */
} /* Pin kontrol fonksiyonunu bitirir. */
int main(void) /* Programin giris noktasini tanimlar. */
{ /* Ana fonksiyon govdesini baslatir. */
    CommandParser parser = {0}; /* Mesaj durumunu bos olarak baslatir. */
    uint8_t byte = 0U; /* Alinan tek UART baytini tutar. */
    uint8_t command[2] = {0}; /* Dogrulanmis iki pin komutunu tutar. */
    uint32_t lastByte = 0U; /* Son karakterin alinma zamanini tutar. */
    HAL_Init(); /* HAL'i, SysTick'i ve temel donanim durumunu baslatir. */
    SystemClock_Config(); /* 16 MHz dahili HSI saatini secer. */
    MX_GPIO_Init(); /* PA5 ve PA6 pinlerini LOW cikis olarak baslatir. */
    MX_USART2_UART_Init(); /* USART2'yi 115200 baud ve 8N1 ile baslatir. */
    while (1) /* Komutlari surekli dinleyen donguyu baslatir. */
    { /* Dongu govdesini baslatir. */
        HAL_StatusTypeDef status = HAL_UART_Receive(&huart2, &byte, 1U, 10U); /* Bir bayti en fazla 10
ms bekler. */
        if (((parser.length != 0U) || (parser.invalid != 0U)) && ((uint32_t)(HAL_GetTick() - lastByte) >
 1000U)) /* Bekleyen mesaji 1 saniye boslukta iptal eder. */
        { /* Zaman asimi dalini baslatir. */
            Command_Reset(&parser); /* Eski kismi mesajin yeni mesajla birlesmesini onler. */
        } /* Zaman asimi dalini bitirir. */
        if (status == HAL_OK) /* Bayt basariyla alindiysa onu isler. */
        { /* Alim dalini baslatir. */
            lastByte = HAL_GetTick(); /* Son alinma zamanini gunceller. */
            if (Command_Feed(&parser, byte, command) != 0U) /* Tam ve gecerli mesaji denetler. */
            { /* Komut uygulama blogunu baslatir. */
                ApplyPin(GPIO_PIN_5, command[0]); /* Ilk karakteri birinci cikis PA5'e uygular. */
                ApplyPin(GPIO_PIN_6, command[1]); /* Ikinci karakteri ikinci cikis PA6'ya uygular. */
            } /* Komut uygulama blogunu bitirir. */
        } /* Alim dalini bitirir. */
        else if (status != HAL_TIMEOUT) /* Normal bos bekleme disindaki UART hatasini ele alir. */
        { /* UART hata dalini baslatir. */
            __HAL_UART_CLEAR_OREFLAG(&huart2); /* SR ve DR okunarak alinma hata bayraklarini temizler.
*/
            Command_Reset(&parser); /* Hatadan etkilenmis mesaji iptal eder. */
            parser.invalid = 1U; /* Sonraki satir sonuna kadar karakterleri reddeder. */
        } /* UART hata dalini bitirir. */
    } /* Sonsuz donguyu bitirir. */
} /* Ana fonksiyonu bitirir. */
static void SystemClock_Config(void) /* Sistem saati ayarlarini uygular. */
{ /* Saat fonksiyonunu baslatir. */
    RCC_OscInitTypeDef osc = {0}; /* Osilator ayarlarini sifirlar. */
    RCC_ClkInitTypeDef clk = {0}; /* Veri yolu saat ayarlarini sifirlar. */
    osc.OscillatorType = RCC_OSCILLATORTYPE_HSI; /* Dahili HSI osilatorunu secer. */
    osc.HSIState = RCC_HSI_ON; /* HSI osilatorunu acar. */
    osc.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT; /* Fabrika kalibrasyonunu kullanir. */
    osc.PLL.PLLState = RCC_PLL_OFF; /* Bu uygulama icin PLL'i kapali tutar. */
    if (HAL_RCC_OscConfig(&osc) != HAL_OK) Error_Handler(); /* Osilator hatasinda programi durdurur. */
    clk.ClockType = RCC_CLOCKTYPE_SYSCLK | RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_PCLK1 |
RCC_CLOCKTYPE_PCLK2; /* Dort saat alanini yapilandirir. */
    clk.SYSCLKSource = RCC_SYSCLKSOURCE_HSI; /* Sistem saatinin kaynagini HSI yapar. */
    clk.AHBCLKDivider = RCC_SYSCLK_DIV1; /* AHB'yi 16 MHz yapar. */
```

```
65      clk.APB1CLKDivider = RCC_HCLK_DIV1; /* APB1'i 16 MHz yapar. */
66      clk.APB2CLKDivider = RCC_HCLK_DIV1; /* APB2'yi 16 MHz yapar. */
67      if (HAL_RCC_ClockConfig(&clk, FLASH_LATENCY_0) != HAL_OK) Error_Handler(); /* Saat ayarini uygular
    ve hatayi denetler. */
68  } /* Saat fonksiyonunu bitirir. */
69  static void MX_GPIO_Init(void) /* Iki GPIO cikisini yapilandirir. */
70  { /* GPIO fonksiyonunu baslatir. */
71      GPIO_InitTypeDef gpio = {0}; /* GPIO ayar yapisini sifirlar. */
72      __HAL_RCC_GPIOA_CLK_ENABLE(); /* GPIOA donaniminin saatini acar. */
73      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6, GPIO_PIN_RESET); /* Cikisa gecmeden iki mandali
    LOW yapar. */
74      gpio.Pin = GPIO_PIN_5 | GPIO_PIN_6; /* PA5 ve PA6 pinlerini secer. */
75      gpio.Mode = GPIO_MODE_OUTPUT_PP; /* Push-pull dijital cikis modunu secer. */
76      gpio.Pull = GPIO_NOPULL; /* Dahili pull-up ve pull-down direncini kapatir. */
77      gpio.Speed = GPIO_SPEED_FREQ_LOW; /* Dusuk hizli GPIO anahtarlamasini secer. */
78      HAL_GPIO_Init(GPIOA, &gpio); /* Secilen GPIO ayarlarini uygular. */
79  } /* GPIO fonksiyonunu bitirir. */
80  static void MX_USART2_UART_Init(void) /* USART2 seri haberlesmesini yapilandirir. */
81  { /* UART fonksiyonunu baslatir. */
82      huart2.Instance = USART2; /* Kullanilacak USART donanimini secer. */
83      huart2.Init.BaudRate = 115200U; /* Saniyedeki sembol sayisini belirler. */
84      huart2.Init.WordLength = UART_WORDLENGTH_8B; /* Veri uzunlugunu 8 bit yapar. */
85      huart2.Init.StopBits = UART_STOPBITS_1; /* Bir stop biti secer. */
86      huart2.Init.Parity = UART_PARITY_NONE; /* Parite denetimini kapatir. */
87      huart2.Init.Mode = UART_MODE_TX_RX; /* Gonderme ve alma yonlerini acar. */
88      huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE; /* RTS/CTS akis kontrolunu kapatir. */
89      huart2.Init.OverSampling = UART_OVERSAMPLING_16; /* 16 kat ornekleme secer. */
90      if (HAL_UART_Init(&huart2) != HAL_OK) Error_Handler(); /* UART'i baslatir ve hatayi denetler. */
91  } /* UART fonksiyonunu bitirir. */
92  static void Error_Handler(void) /* Baslatma hatasinda guvenli durumu uygular. */
93  { /* Hata fonksiyonunu baslatir. */
94      __HAL_RCC_GPIOA_CLK_ENABLE(); /* Cikislari sifirlamak icin GPIOA saatini acar. */
95      HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5 | GPIO_PIN_6, GPIO_PIN_RESET); /* Cikis mandallarini LOW yapar.
    */
96      __disable_irq(); /* Kesmeleri devre disi birakir. */
97      while (1) {} /* Hata sonrasi calismayi durdurur. */
98  } /* Hata fonksiyonunu bitirir. */
```

Core/Src/stm32f4xx_hal_msp.c

```c
#include "stm32f4xx_hal.h" /* HAL donanim turlerini ve fonksiyonlarini ekler. */
void HAL_MspInit(void) /* HAL'in genel alt seviye baslatma kancasini tanimlar. */
{ /* Genel baslatma govdesini baslatir. */
    __HAL_RCC_SYSCFG_CLK_ENABLE(); /* Sistem yapilandirma saatini acar. */
    __HAL_RCC_PWR_CLK_ENABLE(); /* Guc kontrol donanimi saatini acar. */
    HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_4); /* NVIC oncelik gruplamasini belirler. */
} /* Genel baslatma fonksiyonunu bitirir. */
void HAL_UART_MspInit(UART_HandleTypeDef *uart) /* HAL UART baslatmasinin donanim kancasini tanimlar. */
{ /* UART donanim baslatmasini baslatir. */
    GPIO_InitTypeDef gpio = {0}; /* Pin ayar yapisini sifirlar. */
    if (uart->Instance == USART2) /* Yalnizca USART2 icin ayar yapar. */
    { /* USART2 dalini baslatir. */
        __HAL_RCC_USART2_CLK_ENABLE(); /* USART2 cevre birimi saatini acar. */
        __HAL_RCC_GPIOA_CLK_ENABLE(); /* UART pinlerinin GPIOA saatini acar. */
        gpio.Pin = GPIO_PIN_2 | GPIO_PIN_3; /* PA2 TX ve PA3 RX pinlerini secer. */
        gpio.Mode = GPIO_MODE_AF_PP; /* Alternatif fonksiyon push-pull modunu secer. */
        gpio.Pull = GPIO_PULLUP; /* Bos seri hatta HIGH seviyesi icin pull-up secer. */
        gpio.Speed = GPIO_SPEED_FREQ_VERY_HIGH; /* UART pin hizini ayarlar. */
        gpio.Alternate = GPIO_AF7_USART2; /* AF7 ile pinleri USART2'ye baglar. */
        HAL_GPIO_Init(GPIOA, &gpio); /* UART pin ayarlarini uygular. */
    } /* USART2 dalini bitirir. */
} /* UART donanim baslatmasini bitirir. */
```

Core/Src/stm32f4xx_it.c

```
#include "stm32f4xx_hal.h" /* HAL zaman tabani ve CMSIS bildirimlerini ekler. */
void SysTick_Handler(void) /* Her 1 ms olusan sistem kesmesini tanimlar. */
{ /* Kesme govdesini baslatir. */
    HAL_IncTick(); /* HAL milisaniye sayacini artirir. */
} /* Sistem kesmesini bitirir. */
void HardFault_Handler(void) /* Islemci donanim hata kesmesini tanimlar. */
{ /* Hata kesmesini baslatir. */
    while (1) {} /* Hata halinde hata ayiklayici incelemesi icin bekler. */
} /* Hata kesmesini bitirir. */
```